VULNERABILITY TESTING / LAB REPORT

XSS

Web Application Penetration Testing (WAPT) Internship, Phase 1

IqraSec Academy

Field	Details
Intern Name	Rohit Kushwah
Organization	IqraSec Academy
Internship	Web Application Penetration Testing (WAPT), Phase 1
Topic	Cross-Site Scripting (XSS)
Platform	TryHackMe
Number of Labs	1
Completion Date	20 October 2026

Lab 1: XSS

1. Lab Name

Cross-Site Scripting (XSS) — TryHackMe

Room URL: https://tryhackme.com/room/axss

2. Objective

The objective of this lab was to understand Cross-Site Scripting (XSS), learn its different types, and explore how insecure handling of user input can cause unexpected script execution in a web browser.

I studied reflected XSS, stored XSS, DOM-based XSS, common causes, security implications, context and evasion, and prevention techniques.

3. Vulnerability / Concept

Vulnerability: Cross-Site Scripting (XSS)

Cross-Site Scripting is a web application vulnerability in which untrusted input is handled unsafely, allowing attacker-controlled script to execute in another user's browser.

Types of XSS:

- **Reflected XSS:** User input is included in an immediate server response without appropriate handling.
- **Stored XSS:** Untrusted input is stored by an application and later displayed to users without suitable protection.
- **DOM-Based XSS:** Client-side JavaScript processes untrusted data through an unsafe DOM operation, potentially causing script execution.

Why it happens: Common causes include missing context-aware output encoding, unsafe DOM operations, and improper handling of user-controlled input.

Where it appears: Search fields, URL parameters, comments, profile fields, message forms, and other places where applications process or display user input.

Security implications: Depending on the vulnerability and application, XSS may allow page manipulation, exposure of data accessible to scripts, or unauthorized actions through a victim's browser.

4. Testing Approach

The following steps summarize the topics and practical workflow covered in the room. The supplied screenshot confirms that the room displayed 100% completion.

1. Opened the TryHackMe XSS room.
2. Reviewed the introduction to Cross-Site Scripting.
3. Studied XSS terminology and types.
4. Learned the differences between reflected, stored, and DOM-based XSS.

5. Reviewed common causes of XSS vulnerabilities.
6. Studied the security implications of XSS.
7. Opened the Reflected XSS task.
8. Reviewed how user-controlled input may be reflected in a web response.
9. Studied the first vulnerable web application exercise.
10. Reviewed how the application handled user input.
11. Opened the Stored XSS task.
12. Studied how untrusted data may be saved and displayed later.
13. Reviewed the second vulnerable web application exercise.
14. Opened the DOM-Based XSS task.
15. Studied how client-side JavaScript can process untrusted data.
16. Reviewed the context in which data is inserted into a page.
17. Studied the context and evasion concepts covered in the room.
18. Reviewed defensive techniques for preventing XSS.
19. Completed the room's learning tasks and practical exercises.
20. Verified that the TryHackMe room displayed **100% completion**.

Payload / HTTP Request Used, if any:

[Insert the actual payload or request used during testing. Do not add a payload that was not used.]

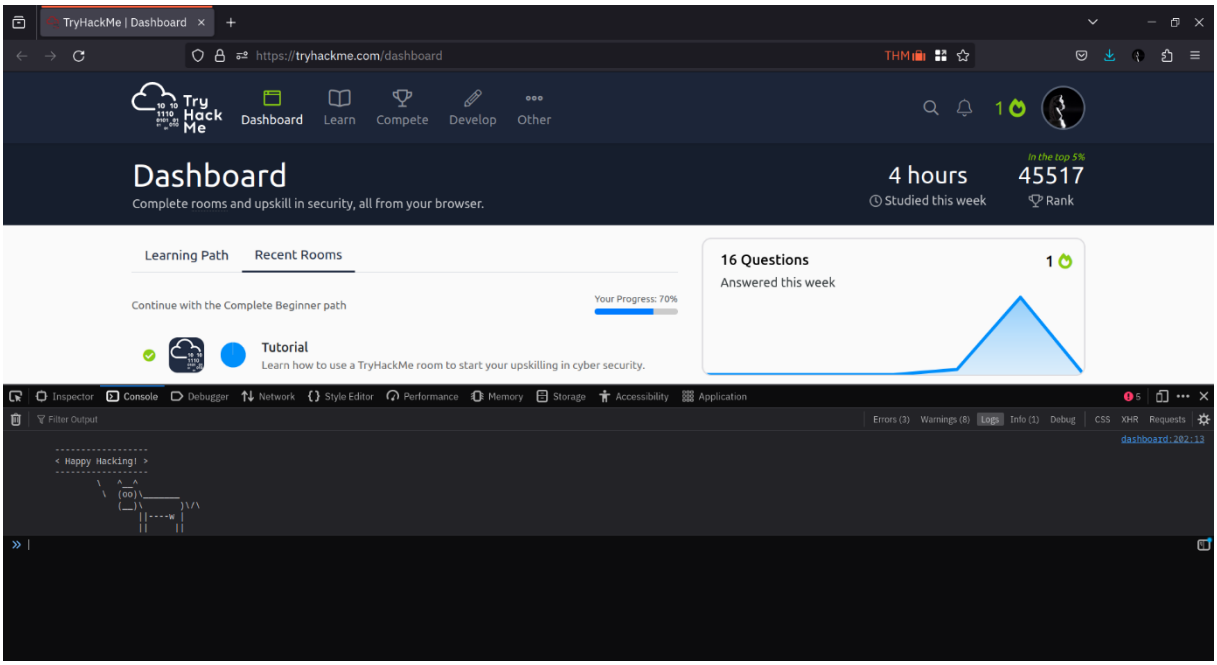
5. Evidence / Screenshots

JavaScript for XSS

Basic knowledge of JavaScript is pivotal for understanding XSS exploits and adapting them to your needs. Knowing that XSS is a client-side attack that takes place on the target's web browser, we should try our attacks on a browser similar to that of the target. It is worth noting that different browsers process certain code snippets differently. In other words, one exploit code might work against Google Chrome but not against Mozilla Firefox or Safari.

Suppose you want to experiment with some JavaScript code in your browser. In that case, you need to open the **Console** found under Web Developer Tools on Firefox, Developer Tools on Google Chrome, and Web Inspector on Safari. Alternatively, use the respective shortcuts:

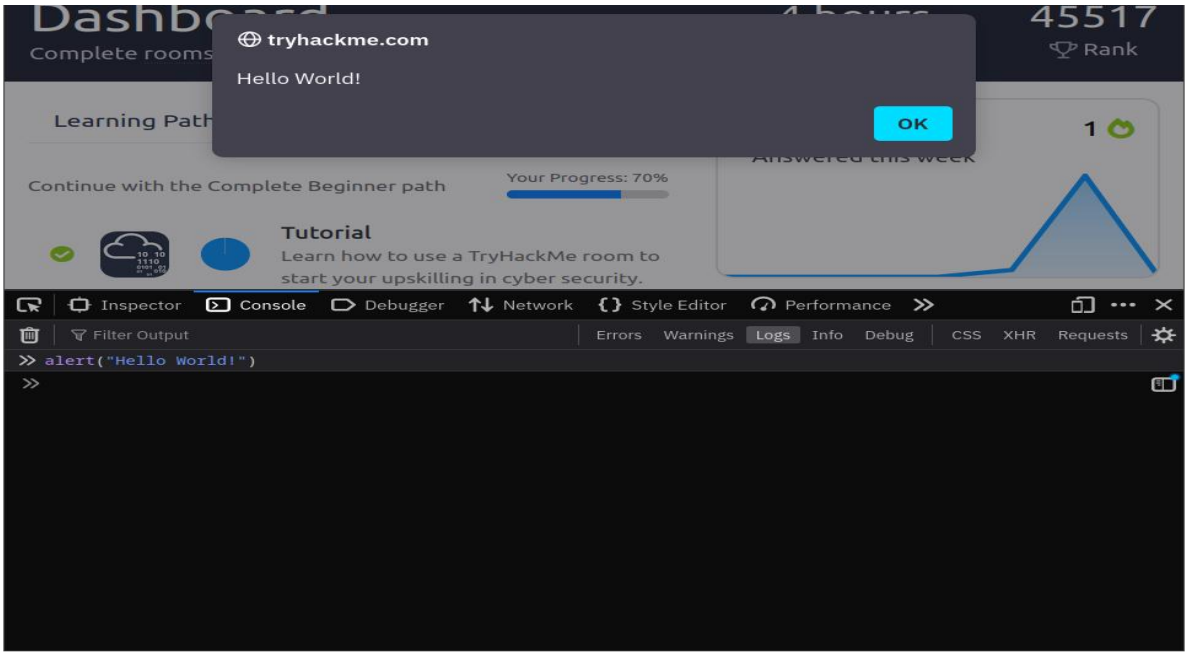
- On Firefox, press Ctrl + Shift + K
- On Google Chrome, press Ctrl + Shift + J
- On Safari, press Command + Option + J

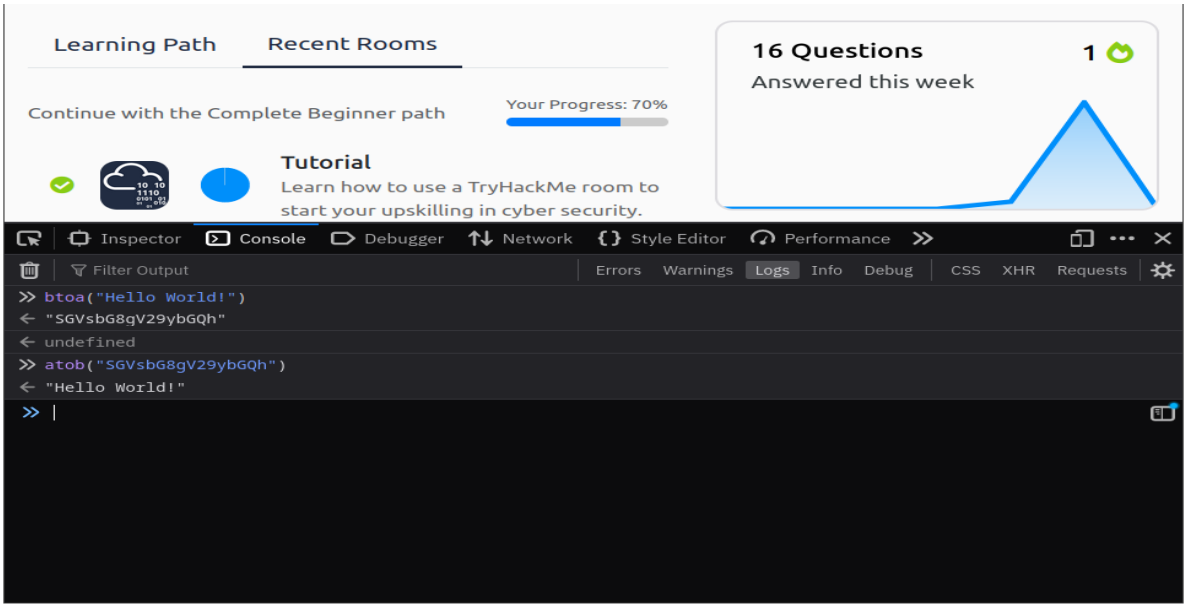


Let's review and try some essential JavaScript functions:

- **Alert**: You can use the alert() function to display a JavaScript alert in a web browser. Try alert(1) or alert('XSS') (or alert("XSS")) to display an alert box with the number 1 or the text XSS.
- **Console log**: Similarly, you can display a value in the browser's JavaScript console using console.log(). Try the following two examples in the console log: console.log(1) and console.log("test text") to display a number or a text string.
- **Encoding**: The JavaScript function btoa("string") encodes a string of binary data to create a base64-encoded ASCII string. This is useful to remove white space and special characters or encode other alphabets. The reverse function is atob("base64_string").

Furthermore, you can experiment with displaying values, such as the document.cookie by using alert(document.cookie) for example.



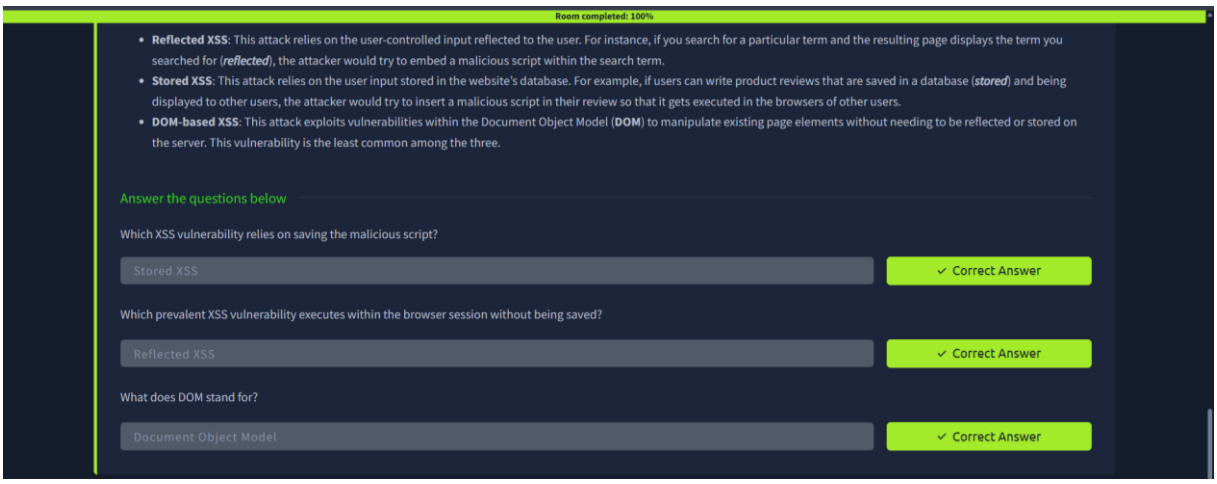


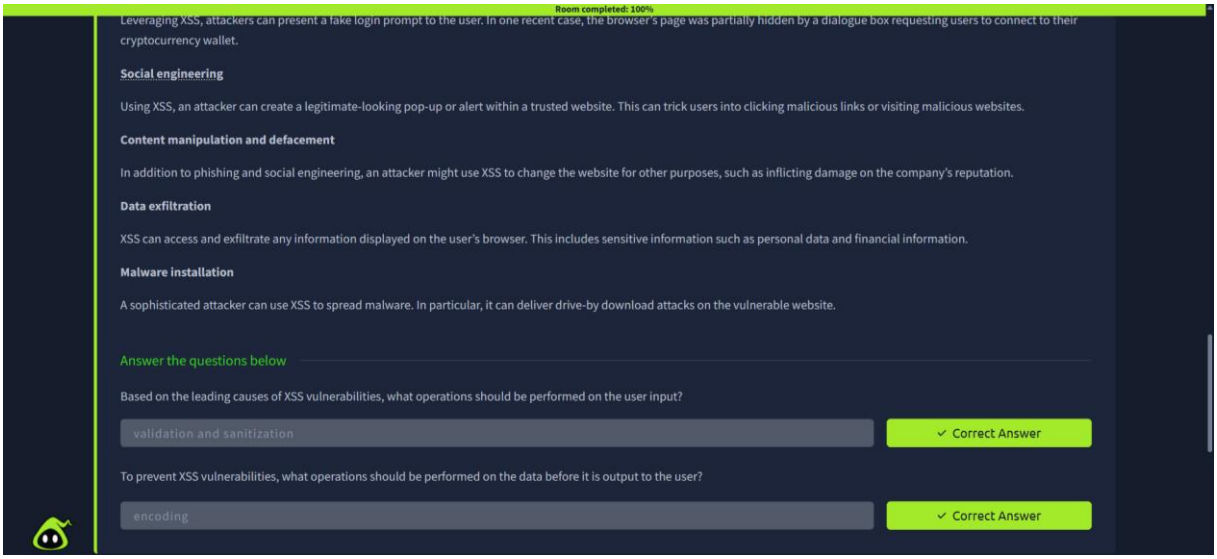
Types of XSS

To recap from the Intro to Cross-site Scripting room, there are three main types of XSS:

- **Reflected XSS**: This attack relies on the user-controlled input reflected to the user. For instance, if you search for a particular term and the resulting page displays the term you searched for (***reflected***), the attacker would try to embed a malicious script within the search term.
- **Stored XSS**: This attack relies on the user input stored in the website's database. For example, if users can write product reviews that are saved in a database (***stored***) and being displayed to other users, the attacker would try to insert a malicious script in their review so that it gets executed in the browsers of other users.

- **DOM-based XSS**: This attack exploits vulnerabilities within the Document Object Model (**DOM**) to manipulate existing page elements without needing to be reflected or stored on the server. This vulnerability is the least common among the three.





Room completed: 100%

PHP

Vulnerable code

Look at the following code snippet in PHP and figure out why it might be vulnerable to reflected XSS.

```
<?php
$search_query = $_GET['q'];
echo "<p>You searched for: $search_query</p>";
?>
```

If you are unfamiliar with PHP, `$_GET` is a PHP array containing values from the URL query string. Furthermore, `$_GET['q']` refers to the query string parameter `q`. For example, in `http://shop.thm/search.php?q=table`, `$_GET['q']` has the value `table`.

As you might have guessed, the vulnerability is caused by the search value displayed on the result page without sanitization. Therefore, an attacker can add a malicious script to the URL, knowing it would be executed. For example, as a proof of concept, the following URL can be tested: `http://shop.thm/search.php?q=<script>alert(document.cookie)</script>` and if the site is vulnerable, an alert box will appear displaying the user's cookie.

Fixed code

Luckily, fixing this code is simple.

```
<?php
$search_query = $_GET['q'];
$escaped_search_query = htmlspecialchars($search_query);
echo "<p>You searched for: $escaped_search_query</p>";
?>
```

JavaScript (Node.js)

Vulnerable code

The following Node.js code snippet is vulnerable to reflected XSS. Try to find the vulnerable part and think of a solution.

```
const express = require('express');
const app = express();

app.get('/search', function(req, res) {
    var searchTerm = req.query.q;
    res.send('You searched for: ' + searchTerm);
});

app.listen(80);
```

If you are unfamiliar with Node.js, the code snippet above uses Express, a popular web application framework for Node.js. The `req.query.q` will extract the value of `q`. For example, in `http://shop.thm/search?q=table`, `req.query.q` has the value `table`. Finally, the response is generated by appending the search term provided by the user to "You searched for:".

Because the value is taken from the user and inserted in the response HTML without sanitization or escaping, it is easy to append a malicious query. As a proof of concept, we can test the following URL: `http://shop.thm/search?q=<script>alert(document.cookie)</script>`, and if the site is vulnerable, an alert box will appear displaying the user's cookie.

Fixed code

```
const express = require('express');
const sanitizeHtml = require('sanitize-html');

const app = express();

app.get('/search', function(req, res) {
    const searchTerm = req.query.q;
    const sanitizedSearchTerm = sanitizeHtml(searchTerm);
    res.send('You searched for: ' + sanitizedSearchTerm);
});

app.listen(80);
```

The solution is achieved by using the `sanitizeHtml()` from the `sanitize-html` library. This function removes unsafe elements and attributes. This includes removing script tags, among other elements that could be used for malicious purposes. You can read its documentation here.

Another approach would be by using the `escapeHtml()` function instead of the `sanitizeHtml()` function. As the name indicates, the `escapeHtml()` function aims to escape characters such as `<`, `>`, `&`, `"`, and `'`. You can check its homepage here.

Python (Flask)

Vulnerable code

Consider the following simple Flask application. Try to find the vulnerable part.

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/search")
def home():
    query = request.args.get("q")
    return f"You searched for: {query}!"

if __name__ == "__main__":
    app.run(debug=True)
```

If you are unfamiliar with Flask, `request.args.get()` is used to access query string parameters from the request URL. In fact, `request.args` contains all the query string parameters in a dictionary-like object. For example, in `http://shop.thm/search?q=table`, `request.args.get("q")` has the value `table`.

Because the value is taken from the user and inserted in the response HTML without sanitization or escaping, it is easy to append a malicious query. As a proof of concept, we can test the following URL: `http://shop.thm/search?q=<script>alert(document.cookie)</script>`, and if the site is vulnerable, an alert box will appear displaying the user's cookie.

Fixed code

```
from flask import Flask, request
from html import escape

app = Flask(__name__)

@app.route("/search")
def home():
    query = request.args.get("q")
    escaped_query = escape(query)
    return f"You searched for: {escaped_query}!"

if __name__ == "__main__":
    app.run(debug=True)
```

The main change is that the user input is now escaped using the `escape()` function from the `html` module. Note that the `html.escape()` function in Flask is actually an alias for `markupsafe.escape()`. They both come from the Werkzeug library and serve the same purpose: escaping unsafe characters in strings. This function converts characters like `<`, `>`, `"`, `'` to HTML escaped entities, disarming any malicious code the user has inserted.

ASP.NET

Vulnerable code

The code snippet below is created using ASP.NET C#.

```
public void Page_Load(object sender, EventArgs e)
{
    var userInput = Request.QueryString["q"];
    Response.Write("User Input: " + userInput);
}
```

If you are unfamiliar with ASP.NET and C#, the code above uses `Request.QueryString`, which returns a collection of associated string keys and values. In the example above, we are interested in the value associated with the key `q`, and we save it in the variable `userInput`. Finally, the response is created by appending the `userInput` to another string.

Fixed code

```
using System.Web;

public void Page_Load(object sender, EventArgs e)
{
    var userInput = Request.QueryString["q"];
    var encodedInput = HttpUtility.HtmlEncode(userInput);
    Response.Write("User Input: " + encodedInput);
}
```

Answer the questions below

Which one of the following characters do you expect to be encoded? `.`, `,`, `;`, `&`, or `#`?

&

✓ Correct Answer

Which one of the following characters do you expect to be encoded? `+`, `-`, `*`, `<`, `=`, or `^`?

<

✓ Correct Answer

Which function can we use in JavaScript to replace (unsafe) special characters with HTML entities?

sanitizeHtml()

✓ Correct Answer

Which function did we use in PHP to replace HTML special characters?

htmlspecialchars()

✓ Correct Answer

Start the attached VM by clicking on the `Start Lab Machine` button to finish this task.

The attached VM runs a vulnerable version of copyparty. The discovered reflected XSS vulnerability has the ID CVE-2023-38501, and its exploit is published here.

The exploit code is `?k304=y%0D%0A%0D%0A%3Cimg+src%3Dcopyparty+onerror%3Dalert(1)%3E` which is the URL encoding of:

```
?k304=y

<img src=copyparty onerror=alert(1)>
```

The attached VM has the vulnerable server running at port 3923. You can reach the vulnerable server at `http://MACHINE_IP:3923` via your AttackBox's browser.

Answer the questions below

What type of vulnerability is it?

Reflected XSS

✓ Correct Answer

Use the above exploit against the attached VM. What do you see on the second line after `go to`?

/?h#cc

✓ Correct Answer

Python (Flask)

Vulnerable Code

Similar to the code snippets we have displayed, the following code uses the Flask framework. By now, you can expect the mistakes plaguing this code.

```python
from flask import Flask, request, render_template_string
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///site.db'
db = SQLAlchemy(app)

class Comment(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    content = db.Column(db.String, nullable=False)

@app.route('/comment', methods=['POST'])
def add_comment():
    comment_content = request.form['comment']
    comment = Comment(content=comment_content)
    db.session.add(comment)
    db.session.commit()
    return 'Comment added!'

@app.route('/comments')
def show_comments():
    comments = Comment.query.all()
    return render_template_string(''.join(['<div>' + c.content + '</div>' for c in comments]))
```

Fixed Code

We are concerned with fixing stored XSS vulnerabilities. We need to ensure that no malicious scripts are saved in the database; furthermore, we will escape any content before displaying it as HTML.

```
from flask import Flask, request, render_template_string, escape
from flask_sqlalchemy import SQLAlchemy
from markupsafe import escape

app = Flask(__name__)
app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///site.db'
db = SQLAlchemy(app)

class Comment(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    content = db.Column(db.String, nullable=False)

@app.route('/comment', methods=['POST'])
def add_comment():
    comment_content = request.form['comment']
    comment = Comment(content=comment_content)
    db.session.add(comment)
    db.session.commit()
    return 'Comment added!'

@app.route('/comments')
def show_comments():
    comments = Comment.query.all()
    sanitized_comments = [escape(c.content) for c in comments]
    return render_template_string(''.join(['<div>' + comment + '</div>' for comment in sanitized_comments]))
```




```
<?php
$con=mysqli_connect("localhost","root","","myhmsdb");
if(isset($_POST['btnSubmit']))
{
    $name = $_POST['txtName'];
    $email = $_POST['txtEmail'];
    $contact = $_POST['txtPhone'];
    $message = $_POST['txtMsg'];

    $query="insert into contact(name,email,contact,message) values('$name','$email','$contact','$message');";

//...
}
```

Answer the questions below

What type of vulnerability is it?

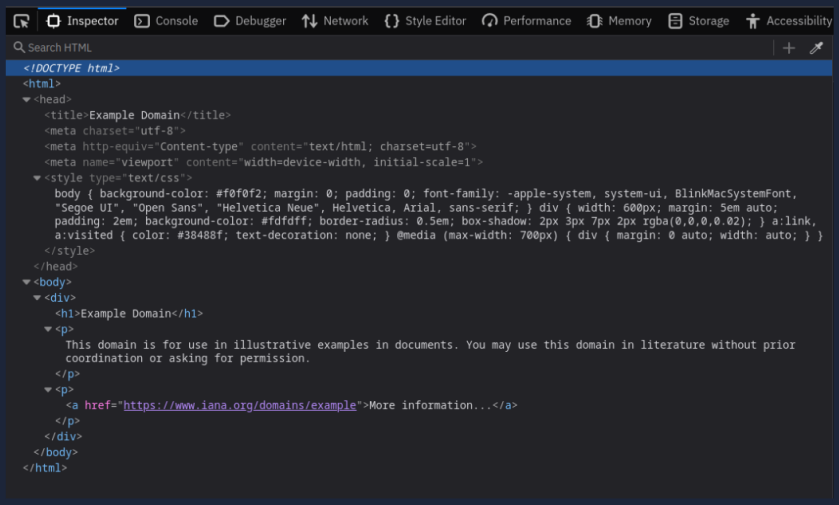
Stored XSS ✓ Correct Answer

Go to the contact page and submit the following message `<script>alert(document.cookie)</script>`. Next, log in as the Receptionist. What is the name of the key from the displayed key-value pair?

PHPSESSID ✓ Correct Answer

Before we dive into DOM-based XSS, let's review what Document Object Model (DOM) is. The DOM is a programming interface representing a web document as a tree. The DOM makes it possible to programmatically access and manipulate the different parts of a website using JavaScript. Let's consider a practical example.

Consider the HTML code of example.com in the screenshot below (as fetched on the first of February 2024). We opened the Web Developer Tools using the Firefox web browser and checked the Inspector tab.



The DOM tree shown above is like the following list with sublists.

</p>

</body>
</html>

The DOM tree shown above is like the following list with sublists.

- `document`
  - `<!DOCTYPE html>`
  - `html`
    - `head`
      - `title`
      - `meta`
      - `meta`
      - `meta`
      - `style`
    - `body`
      - `div`
        - `h1`
        - `p`
        - `p`
          - `a`

The tree starts with the `document` node and branches into `DOCTYPE` and `html`. The `html` node branches into `head` and `body`. The `head` has the `title`, a few `meta` tags, and a `style`. In this simple example, the `body` has a single `div` that branches into one `h1` and two `p`. This is a concise page. More practical pages would have tens or hundreds of branches.

We can view the DOM tree using the web browser's built-in Web Developer's Tools. For example, press Ctrl + Shift + I on Firefox and check the Inspector tab.

Alternatively, we can access the JavaScript console, as mentioned in Task 2. Using JavaScript, you can manipulate the DOM tree. For example, you can create a new element using `document.createElement()` and add a child to any element using `element.append()`. Here is an example from MDN documentation.

Alternatively, we can access the JavaScript console, as mentioned in Task 2. Using JavaScript, you can manipulate the DOM tree. For example, you can create a new element using `document.createElement()` and add a child to any element using `element.append()`. Here is an example from MDN documentation.

```
let div = document.createElement("div");
let p = document.createElement("p");
div.append(p);

console.log(div.childNodes); // NodeList [ <p> ]
```

In the example code above, we created two elements, `div` and `p`. Then, we appended the latter element to the `div` element.

Vulnerable Web Applications

DOM-based XSS vulnerabilities take place within the browser. They don't need to go to the server and return to the client's web browser. In other words, the attacker will try to exploit this situation by injecting a malicious script, for example, into the URL, and it will be executed on the client's side without any role for the server in this. We will present an elementary and minimal static site without relying on the back-end code to demonstrate this concept.

Vulnerable "Static" Site

Let's consider the following basic example. It is too simple that it is unrealistic; however, it is enough to demonstrate the DOM-based XSS.

```
<!DOCTYPE html>
<html>
<head>
    <title>Vulnerable Page</title>
</head>
<body>
    <div id="greeting"></div>
    <script>
        const name = new URLSearchParams(window.location.search).get('name');
```

minimal static site without relying on the back-end code to demonstrate this concept.

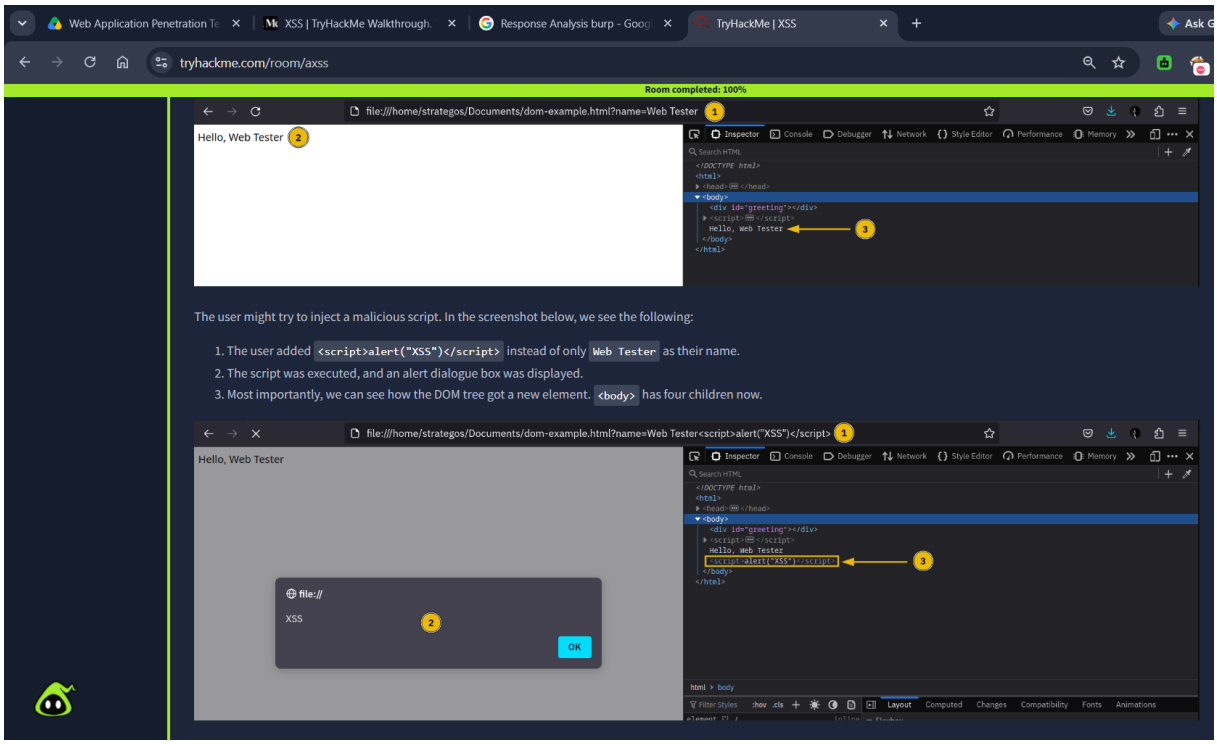
Vulnerable "Static" Site

Let's consider the following basic example. It is too simple that it is unrealistic; however, it is enough to demonstrate the DOM-based XSS.

```
<!DOCTYPE html>
<html>
<head>
    <title>Vulnerable Page</title>
</head>
<body>
    <div id="greeting"></div>
    <script>
        const name = new URLSearchParams(window.location.search).get('name');
        document.write("Hello, " + name);
    </script>
</body>
</html>
```

The page above expects the user to provide their name after `?name=`. In the screenshot below:

1. The user has entered Web Tester after `?name` in the URL.
2. The greeting worked as expected and displayed "Hello, Web Tester".
3. Finally, the DOM structure on the right is left intact; the `<body>` has three direct children.



This basic example illustrates a couple of things:

- The server has no direct role in DOM-based vulnerabilities. In this demonstration, everything took place on the client's browser without using a back end.
- The DOM was insecurely modified using `document.write()`.

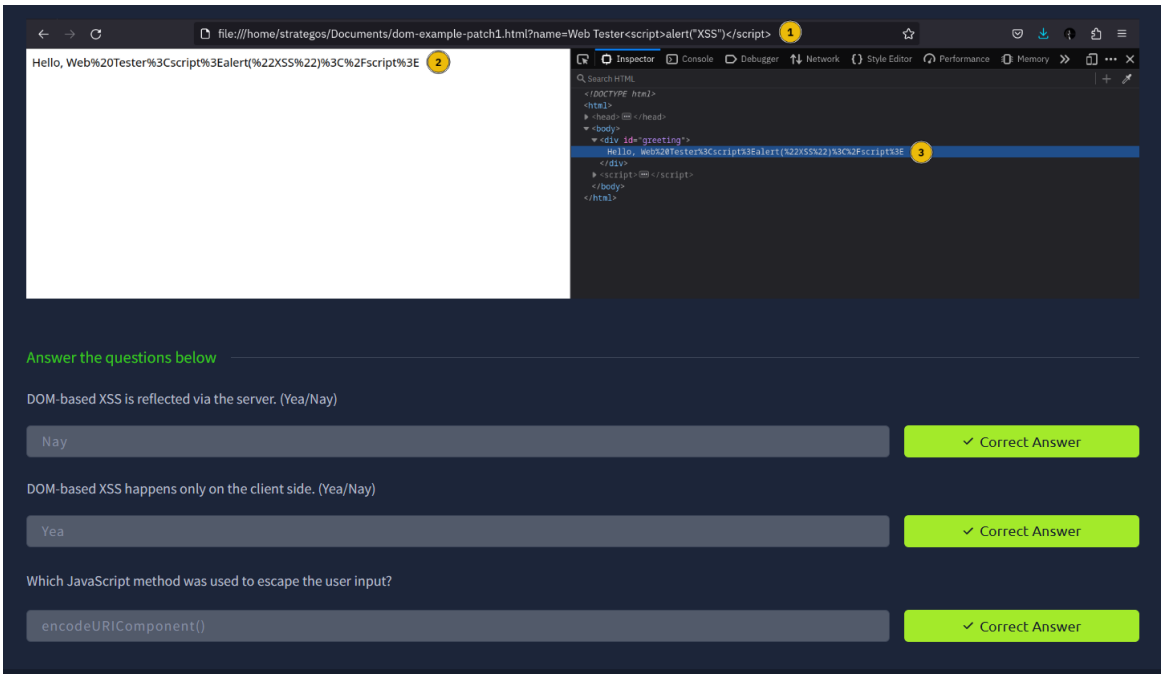
Fixed "Static" Site

```
<!DOCTYPE html>
<html>
<head>
    <title>Secure Page</title>
</head>
<body>
    <div id="greeting"></div>
    <script>
        const name = new URLSearchParams(window.location.search).get('name');
        // Escape the user input to prevent XSS attacks
        const escapedName = encodeURIComponent(name);
        document.getElementById("greeting").textContent = "Hello, " + escapedName;
    </script>
</body>
</html>
```

One way to fix this page is by avoiding adding user input directly with `document.write()`. Instead, we first escaped the user input using `encodeURIComponent()` and then added it to `textContent`.

The previous attempt does not work now. We can see that:

1. The user has added JavaScript as part of their input.
2. The JavaScript code is displayed as encoded characters and presents no threat in the current context.



---

6. Impact

If exploited, XSS may allow an attacker to execute script in a victim's browser within the vulnerable application's origin. Depending on the application, this could result in:

- Manipulation of page content.
- Exposure of data accessible to client-side scripts.
- Unauthorized actions through a victim's active session.
- Phishing content displayed on a trusted website.
- Loss of user trust and reputational damage.

The actual impact depends on the vulnerability, application behavior, user privileges, and security controls. It should not be overstated beyond what testing demonstrates.

7. Remediation

- **Use context-aware output encoding:** Encode untrusted data according to whether it is placed in HTML, an attribute, JavaScript, CSS, or a URL.
- **Use safe DOM APIs:** Prefer textContent for plain text and avoid unsafe insertion of untrusted HTML.
- **Validate input:** Apply suitable allowlists and format checks where appropriate.
- **Sanitize permitted HTML:** Use a reputable HTML sanitizer when the application must accept user-supplied markup.
- **Use secure templates:** Keep automatic output escaping enabled in the framework or template engine.
- **Implement Content Security Policy (CSP):** Use CSP as an additional defense-in-depth control.

- **Test input-handling paths:** Include reflected, stored, and DOM-based XSS in authorized security testing.
- **Protect sessions:** Configure cookies with appropriate HttpOnly, Secure, and SameSite attributes.

8. What You Learned

- I learned what Cross-Site Scripting is and how it affects web applications.
- I understood the differences between reflected, stored, and DOM-based XSS.
- I learned why unsafe handling of user input can cause script execution.
- I understood why the context in which data is inserted affects testing.
- I studied the causes and possible implications of XSS.
- I learned about output encoding, safe DOM APIs, sanitization, and Content Security Policy.
- I understood the importance of collecting genuine evidence and documenting results accurately.

Conclusion: I completed the TryHackMe XSS room, and the supplied screenshot shows 100% completion. The room helped me understand major XSS types, their causes, their potential impact, and the defensive techniques used to reduce these vulnerabilities.